Trabajo Práctico – Google v. Oracle y Viacom v. YouTube

Legislación – Tecnicatura Universitaria en Programación a Distancia – UTN

Alumno: Ignacio Figueroa · **DNI:** 45.406.120

Introducción

En este trabajo analizo dos casos judiciales de Estados Unidos que son muy importantes para la industria del software y de internet. El primero, Google v. Oracle, trata sobre si se puede copiar la estructura de una API para crear un sistema nuevo. El segundo, Viacom v. YouTube, trata sobre si una plataforma tiene la culpa cuando sus usuarios suben contenido protegido por derechos de autor.

Aunque son casos de otro país, me parecen relevantes porque las empresas y herramientas involucradas (Java, Android, YouTube) son las que usamos todos los días como programadores, y porque las reglas que se discutieron ahí terminaron influyendo en cómo funciona internet en todo el mundo. Cuando corresponde, también relaciono los temas con la legislación argentina.

Nota: en la consigna no aparece la pregunta 2, así que respeto la numeración original.

Caso 1: Google v. Oracle (2010–2021)

1. ¿Qué fue exactamente lo que Oracle acusó a Google de haber copiado?

Oracle, que es dueña de Java desde que compró Sun Microsystems, acusó a Google de copiar parte de la **API de Java SE**. Lo que se copió no fue todo el código de Java, sino el **"código de declaración"** (*declaring code*) de 37 paquetes de la API: los nombres de las clases y métodos, sus parámetros y la forma en que están organizados. Por ejemplo, la línea que declara que existe un método `Math.max(int a, int b)`.

Google escribió su propio **código de implementación** (lo que hace cada método por dentro), pero mantuvo los mismos nombres y la misma estructura para que los programadores que ya sabían Java pudieran usarlos igual. Oracle sostenía que esa estructura, secuencia y organización también estaba protegida por derechos de autor.

3. ¿Cuántas líneas de código estaban en disputa aproximadamente?

Unas **11.500 líneas** de código de declaración. Para tener una idea, la Corte Suprema señaló que eso representaba apenas el **0,4%** del total de la API de Java SE, que tenía cerca de 2,86 millones de líneas.

4. ¿Para qué sistema utilizó Google ese código?

Para **Android**, su sistema operativo para celulares. Google quería que los programadores que ya sabían Java pudieran desarrollar apps para Android sin tener que aprender todo de nuevo.

5. ¿Por qué la reutilización de APIs puede ser importante para los desarrolladores?

Porque una API es como un "idioma" común. Cuando un programador aprende a usar una API, aprende nombres de métodos, cómo se llaman y qué devuelven. Si otras plataformas pueden reutilizar esa misma estructura:

- no hay que aprender todo de cero cada vez que se cambia de plataforma;
- se puede reutilizar código que ya existe;
- se desarrolla más rápido y con menos errores;
- hay más competencia, porque un nuevo sistema puede entrar al mercado sin que los programadores tengan que "empezar de nuevo".

En mi caso, por ejemplo, si aprendo Java en la tecnicatura, es una ventaja enorme poder usar lo mismo en Android.

6. ¿Qué significa el concepto de interoperabilidad en software?

La **interoperabilidad** es la capacidad de distintos sistemas o programas de comunicarse y trabajar juntos, intercambiando información sin problemas, aunque hayan sido hechos por empresas distintas. Por ejemplo, que una app pueda conectarse a Mercado Pago, que un sistema web lea datos de una API de clima o que un archivo hecho en un programa se abra en otro. Las APIs son justamente la herramienta principal para lograr esa interoperabilidad.

7. ¿Por qué la Corte Suprema consideró que el uso de las APIs fue 'uso legítimo' (fair use)?

El 5 de abril de 2021 la Corte Suprema falló **6 a 2 a favor de Google**, con el voto escrito por el juez Breyer. Un detalle interesante es que la Corte **no decidió si las APIs tienen o no derechos de autor**: lo dejó de lado y dijo que, aun suponiendo que los tuvieran, el uso de Google era legítimo. Para eso analizó los cuatro factores del fair use:

1. **Propósito y carácter del uso:** Google hizo un uso **transformador**, porque usó esas declaraciones para crear una plataforma nueva (celulares), distinta del uso original de Java en computadoras.
2. **Naturaleza de la obra:** el código de declaración es una interfaz funcional cuyo valor viene en gran parte de que los programadores la aprendieron, así que está más lejos del "núcleo" de lo que protege el derecho de autor.
3. **Cantidad copiada:** solo se copió lo necesario para que los programadores pudieran usar sus conocimientos, y era el 0,4% del total.
4. **Efecto en el mercado:** Android no reemplazó a Java en el mercado de computadoras, y de hecho ayudó a que más programadores usaran Java. Además, darle a Oracle un control total sobre la API le hubiera permitido bloquear la innovación de otros.

Dos jueces (Thomas y Alito) votaron en contra, argumentando que el código de declaración sí tenía protección y que la copia había afectado el mercado de Oracle.

8. ¿Qué problemas podría generar para la industria del software si las APIs fueran completamente propietarias?

- **Dependencia de una sola empresa (lock-in):** los programadores y las empresas quedarían atados a quien tiene los derechos de la API.
- **Menos competencia:** sería muy difícil crear alternativas compatibles con un sistema existente.
- **Más costos:** habría que pagar licencias solo por usar una forma de comunicarse entre sistemas.
- **Riesgo legal constante:** muchas cosas que hoy son normales (emuladores, librerías compatibles, sistemas open source que imitan otros) podrían terminar en juicios.
- **Freno a la innovación:** cada nuevo proyecto tendría que reinventar la rueda.

Muchas empresas y proyectos open source se basan en reimplementar APIs existentes, así que el impacto hubiera sido enorme.

9. Imaginá que estás desarrollando una aplicación que necesita interactuar con otra plataforma. ¿Qué ventajas te da poder usar APIs existentes?

Si, por ejemplo, armo una app de turnos que tiene que mandar recordatorios por WhatsApp, cobrar con Mercado Pago y mostrar la ubicación en Google Maps, las APIs me dan:

- **Ahorro de tiempo:** no tengo que programar desde cero un sistema de pagos o de mapas.

- **Confiabilidad:** uso servicios que ya están probados por millones de usuarios.
- **Integración:** mi app puede comunicarse con otros sistemas que la gente ya usa.
- **Foco en lo importante:** me concentro en la lógica de mi aplicación y no en problemas que otros ya resolvieron.
- **Documentación y comunidad:** hay ejemplos y gente que ya pasó por los mismos problemas.

Obviamente tengo que respetar los términos de uso de cada API, que es algo distinto a copiar su estructura.

10. ¿Creés que copiar estructuras de APIs debería ser legal? Justificá tu respuesta.

Sí, creo que debería ser legal, pero con límites. Para mí, la estructura de una API (los nombres de los métodos y cómo se organizan) funciona más como un estándar o un idioma que como una obra creativa. Si alguien pudiera adueñarse de eso, sería como si alguien tuviera derechos sobre la forma de enchufar un cable o sobre el orden de las teclas de un teclado.

Dicho esto, no me parece bien que se copie el **código de implementación**, que es donde realmente está el trabajo y la creatividad del programador. Google escribió el suyo, y eso para mí es lo que marca la diferencia. Tampoco me parece justo copiar algo solo para competir sin aportar nada nuevo. En resumen: copiar la interfaz para lograr compatibilidad sí; copiar el código que hace funcionar las cosas, no.

Caso 2: Viacom v. YouTube (2007–2012)

11. ¿Por qué Viacom demandó a YouTube?

Viacom, un gran grupo de medios dueño de canales como MTV, Comedy Central y Nickelodeon, demandó a YouTube (y a Google, que lo había comprado en 2006) en marzo de 2007 por **infracción de derechos de autor**, tanto directa como indirecta. Viacom sostenía que en YouTube había miles de videos con su contenido subidos sin autorización, que YouTube sabía que eso pasaba y que incluso se beneficiaba, porque ese contenido atraía usuarios y publicidad. Según Viacom, YouTube había construido su negocio en base a la piratería.

12. ¿Qué tipo de contenido estaba en conflicto?

Videos de **programas de televisión** de Viacom subidos por los usuarios sin permiso: fragmentos o episodios de series y shows como *South Park*, *The Daily Show* o *Bob Esponja*, entre otros. En la demanda se mencionaban decenas de miles de clips (se habló de

alrededor de 79.000 en la etapa final del juicio, y de más de 150.000 en las primeras versiones).

13. ¿Qué monto reclamaba Viacom en la demanda?

Más de 1.000 millones de dólares en daños y perjuicios, además de que se ordenara a YouTube dejar de permitir la infracción.

14. ¿Qué ley fue clave en la resolución del caso?

La **DMCA** (*Digital Millennium Copyright Act*), una ley de Estados Unidos de 1998, en particular su **sección 512(c)**, que establece el "puerto seguro" para los sitios que alojan contenido subido por usuarios.

El recorrido del caso fue largo:

Año	Qué pasó
2007	Viacom presenta la demanda
2010	El juez de primera instancia (Louis Stanton) falla a favor de YouTube: está protegido por el puerto seguro
2012	La Cámara de Apelaciones del Segundo Circuito revoca en parte y devuelve el caso para revisar si YouTube sabía de infracciones específicas
2013	El mismo juez vuelve a fallar a favor de YouTube
2014	Viacom y Google llegan a un acuerdo y terminan el juicio

15. ¿Qué es el 'puerto seguro' (safe harbor) en la DMCA?

Es una protección legal que dice que un proveedor de servicios en internet (como YouTube, una red social o un hosting) **no es responsable** por las infracciones de derechos de autor que cometen sus usuarios, siempre que cumpla ciertas condiciones. La idea es que la plataforma es como un "intermediario": no elige el contenido, solo lo aloja. Mientras actúe de buena fe y colabore cuando le avisan de una infracción, queda "a salvo" de ser demandada por eso.

16. ¿Qué condiciones debe cumplir una plataforma para no ser responsable del contenido subido por los usuarios?

Según la DMCA, la plataforma tiene que:

1. **No tener conocimiento efectivo** de que un contenido específico infringe derechos, ni conocer hechos que lo hagan evidente (lo que se llama *red flag knowledge*).
2. **Actuar rápido** para eliminar o bloquear el contenido apenas se entera de la infracción o recibe una notificación.
3. **No obtener un beneficio económico directo** de la infracción cuando tiene el derecho y la capacidad de controlarla.
4. **Designar un agente** para recibir las notificaciones de infracción y registrarlo.
5. **Tener una política contra infractores reincidentes**, por ejemplo cerrando las cuentas de quienes suben contenido ilegal una y otra vez.
6. **No interferir** con las medidas técnicas que usan los titulares para identificar o proteger sus obras.

Un punto clave del caso fue que tener un **conocimiento general** de que hay piratería en el sitio no alcanza para perder la protección. Lo que importa es saber de **infracciones específicas** y no hacer nada.

17. ¿Qué rol juega la notificación de infracción (takedown notice)?

Es el mecanismo central del sistema. Funciona así:

1. El titular de los derechos detecta un contenido que infringe su obra.
2. Le envía a la plataforma una **notificación formal** identificando el contenido y declarando que tiene los derechos.
3. La plataforma **lo baja rápidamente**.
4. El usuario que lo subió puede presentar una **contranotificación** si cree que tenía derecho a publicarlo, y en ese caso el contenido puede volver a subirse.

De esta forma, la carga de vigilar recae principalmente en el titular de los derechos, que es quien sabe qué es suyo, y la plataforma tiene que reaccionar. En el caso, antes de la demanda Viacom había mandado más de 100.000 notificaciones y YouTube las había cumplido, lo que ayudó a su defensa.

18. ¿Qué riesgos existirían si las plataformas fueran responsables automáticamente por todo el contenido que suben los usuarios?

- **Censura previa:** las plataformas tendrían que revisar todo antes de publicarlo y, ante la duda, borrarían mucho contenido legítimo (parodias, críticas, reseñas, videos propios).
- **Afectación a la libertad de expresión:** menos gente podría publicar libremente.

- **Costos enormes:** solo las empresas gigantes podrían pagar sistemas de control y abogados; las plataformas chicas o nuevas directamente no podrían existir.
- **Menos innovación:** casi nadie se animaría a crear una red social, un foro o un repositorio de código.
- **Imposibilidad práctica:** en YouTube se suben cientos de horas de video por minuto, así que revisar todo manualmente es imposible.

19. Si crearas una plataforma tipo red social o repositorio de código: ¿qué medidas implementarías para cumplir con la ley?

- **Términos y condiciones claros** que prohíban subir contenido o código sin tener los derechos.
- **Un formulario de denuncia** fácil de usar para que los titulares notifiquen infracciones, y un responsable o agente designado para atenderlas.
- **Un procedimiento de baja rápida** y la posibilidad de que el usuario se defienda (contranotificación).
- **Una política de reincidentes:** advertencias y, si se repite, suspensión de la cuenta.
- **Detección automática** de contenido protegido, en la medida de lo posible (parecido al Content ID de YouTube), aunque con revisión humana para evitar errores.
- En el caso de un repositorio de código: **detección de licencias** y aviso cuando alguien sube código con licencias incompatibles, además de mostrar la licencia de cada proyecto.
- **Transparencia:** publicar cuántas denuncias se reciben y cómo se resuelven.
- **Adaptarme a la ley local.** En Argentina no hay una ley como la DMCA, pero la Corte Suprema, en el caso *Rodríguez, María Belén c/ Google* (2014), estableció que los intermediarios responden solo si tienen **conocimiento efectivo** del contenido ilícito y no actúan para bajarlo. Es un criterio parecido al del puerto seguro.

20. ¿Creés que el sistema actual favorece más a las empresas tecnológicas o a los creadores de contenido? Justificá.

En mi opinión, favorece más a las **empresas tecnológicas**, aunque no completamente. El puerto seguro les permitió crecer sin hacerse cargo de todo lo que suben los usuarios, y eso es lo que hizo posible que existan YouTube o GitHub. El caso Viacom lo muestra: aunque YouTube tenía mucho contenido infractor, ganó.

Por otro lado, a los creadores les toca la tarea de rastrear su contenido y mandar notificaciones, lo cual es mucho trabajo, sobre todo para creadores chicos que no tienen un equipo legal. Las grandes empresas de medios se defienden mejor, en parte porque

hoy tienen herramientas como Content ID.

Igual, creo que el sistema también tiene un costado bueno para los creadores: gracias a estas plataformas, millones de personas pueden publicar su contenido y ganar plata sin depender de un canal de TV o una discográfica. Así que más que "empresas vs. creadores", diría que favorece a las plataformas grandes y a los titulares grandes, y los que quedan más desprotegidos son los creadores chicos de ambos lados.

Preguntas integradoras

21. Compará ambos casos: ¿qué tienen en común en términos de propiedad intelectual y en qué se diferencian?

	Google v. Oracle	Viacom v. YouTube
Qué se protegía	Código de software (la API de Java)	Contenido audiovisual (programas de TV)
Quién copió	Google, directamente	Los usuarios de YouTube
Tipo de responsabilidad discutida	Directa: ¿Google infringió?	Indirecta: ¿YouTube responde por lo que hicieron sus usuarios?
Figura legal clave	Fair use (uso legítimo)	Safe harbor de la DMCA (puerto seguro)
Resultado	Ganó Google en la Corte Suprema (2021)	Ganó YouTube en primera instancia y luego hubo un acuerdo (2014)

En común: los dos casos tratan sobre derechos de autor en el mundo digital y enfrentan a un titular de derechos con una empresa tecnológica que creó algo nuevo usando contenido ajeno. En ambos, Google terminó ganando, y en ambos la justicia buscó un equilibrio para no frenar la innovación. También en los dos aparece la misma pregunta de fondo: hasta dónde llega el derecho de autor cuando se cruza con la tecnología.

Diferencias: en Oracle, Google copió algo por decisión propia y se discutió si eso era legítimo. En Viacom, YouTube no copió nada directamente, sino que se discutía si tenía que hacerse cargo de lo que subían otros. Además, uno trata sobre código (algo más funcional) y el otro sobre obras artísticas (algo más creativo).

22. ¿Cómo influyen estos fallos en el trabajo diario de un programador?

Más de lo que parece:

- **Gracias a Google v. Oracle,** puedo usar e implementar APIs conocidas con más tranquilidad, crear librerías compatibles o migrar de una plataforma a otra sin miedo a que me demanden por usar los mismos nombres de métodos. Eso sí, el fallo no dice que se pueda copiar cualquier código: el código de implementación sigue protegido.
- **Gracias al safe harbor,** existen plataformas como GitHub, Stack Overflow o npm, donde todos los días uso código y respuestas que suben otras personas. Sin esa protección probablemente no existirían o serían mucho más cerradas.
- **Me obligan a prestar atención a las licencias:** antes de copiar un fragmento de código o usar una librería, tengo que fijarme qué licencia tiene (MIT, GPL, Apache, etc.) y qué me permite hacer.
- **Si desarrollo una plataforma** donde los usuarios suben contenido, tengo que pensar desde el principio en un sistema de denuncias y bajas.

En Argentina, el software está protegido por la **Ley 11.723 de Propiedad Intelectual** (que lo incluye expresamente desde la reforma de la Ley 25.036 de 1998), así que el respeto por el código ajeno también es una obligación legal acá.

23. ¿Qué equilibrio debería existir entre innovación tecnológica, derechos de autor y acceso al conocimiento?

Creo que el derecho de autor tiene que **proteger el esfuerzo creativo** para que la gente tenga incentivos para crear, pero sin convertirse en una herramienta para bloquear a otros. El equilibrio que yo propondría sería:

- Proteger fuerte lo que es realmente creativo y costoso de hacer (el código de implementación, una película, un libro).
- Dejar libres los elementos que funcionan como estándares o "idiomas" comunes (APIs, formatos de archivo, protocolos), porque sin ellos no hay interoperabilidad.
- Mantener excepciones para la educación, la investigación, la crítica y la parodia.
- Que las plataformas colaboren con los titulares, pero sin tener que censurar todo por las dudas.

Como estudiante, el acceso al conocimiento me importa mucho: gran parte de lo que aprendí de programación fue con documentación abierta, código open source y videos gratuitos. Si todo estuviera cerrado, aprender sería mucho más difícil y caro.

24. ¿Creés que las leyes actuales están actualizadas frente a la realidad del desarrollo de software? ¿Por qué?

No del todo. Muchas de las leyes que se aplican fueron pensadas para libros, música o películas y se fueron adaptando al software. La DMCA es de 1998, cuando no existían YouTube ni las redes sociales, y la Ley 11.723 en Argentina es de 1933, aunque se le agregó el software en 1998.

Una muestra de que las leyes van detrás de la tecnología es que en el caso Oracle la Corte Suprema ni siquiera quiso definir si las APIs tienen derechos de autor, y lo resolvió por otro camino. Otra muestra es el problema actual con la **inteligencia artificial**: por ejemplo, en diciembre de 2023 el New York Times demandó a OpenAI y Microsoft por usar millones de sus artículos para entrenar a ChatGPT sin permiso. No hay una ley que diga claramente si eso es legal, y se está discutiendo caso por caso, igual que pasó con las APIs y con YouTube en su momento.

En Argentina, además, no tenemos una ley específica sobre la responsabilidad de las plataformas; eso lo fue definiendo la jurisprudencia, como en el caso *Rodríguez c/ Google*.

25. Proponé una mejora o cambio en la legislación que beneficie tanto a desarrolladores como a empresas.

Mi propuesta sería una **ley de interoperabilidad y responsabilidad de plataformas** con tres puntos:

1. **Excepción expresa para interfaces:** que la ley diga claramente que copiar o reimplementar la estructura de una API, un protocolo o un formato de archivo para lograr compatibilidad no infringe derechos de autor, siempre que el código de implementación sea propio. Así los desarrolladores no dependen de que un juez interprete el fair use, y las empresas saben a qué atenerse.
2. **Un sistema de notificación y baja con reglas claras,** parecido a la DMCA pero adaptado a Argentina: plazos definidos para bajar el contenido, derecho del usuario a defenderse y sanciones para quien haga denuncias falsas o abusivas para sacar contenido legítimo.
3. **Reglas claras para el uso de obras en el entrenamiento de inteligencia artificial,** por ejemplo permitiendo que los autores se excluyan (*opt-out*) o que reciban una compensación, para que las empresas puedan innovar con seguridad jurídica y los creadores no queden desprotegidos.

Creo que esto beneficiaría a los dos lados: los desarrolladores tendrían más libertad para crear cosas compatibles y las empresas tendrían reglas previsibles, que es lo que más necesitan para invertir.

Fuentes consultadas

- Resumen de Google LLC v. Oracle America (2021) – U.S. Copyright Office
- Google v. Oracle: la Corte Suprema falla a favor de Google – Jones Day
- Viacom v. YouTube – Electronic Frontier Foundation
- Viacom v. YouTube: los límites del safe harbor – JURIST
- Acuerdo final entre Viacom y Google (2014) – DWW
- Video de la cátedra: demanda del New York Times contra OpenAI y Microsoft
- Ley 11.723 de Propiedad Intelectual (Argentina) y Ley 25.036.
- CSJN, *Rodríguez, María Belén c/ Google Inc. s/ daños y perjuicios* (2014).